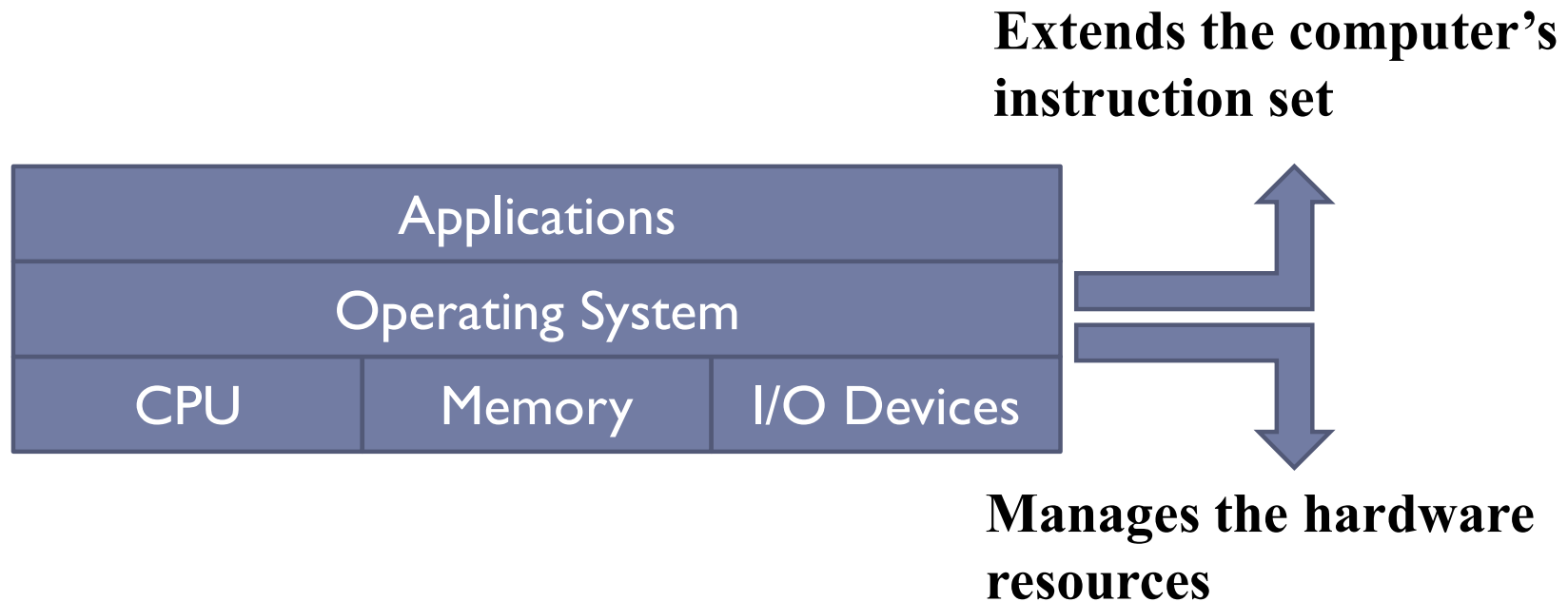


Computer System Overview

计算机系统概述

Chapter 1

Why should we understand the hardware?



Main Contents

- ▶ **Basic Elements of a Computer System**

- ▶ **Processor**

- ▶ Processor Registers
 - ▶ Instruction Execution
 - ▶ Interrupts

- ▶ **Memory**

- ▶ Memory Hierarchy
 - ▶ Cache Memory (hit ratio)

- ▶ **I/O Device**

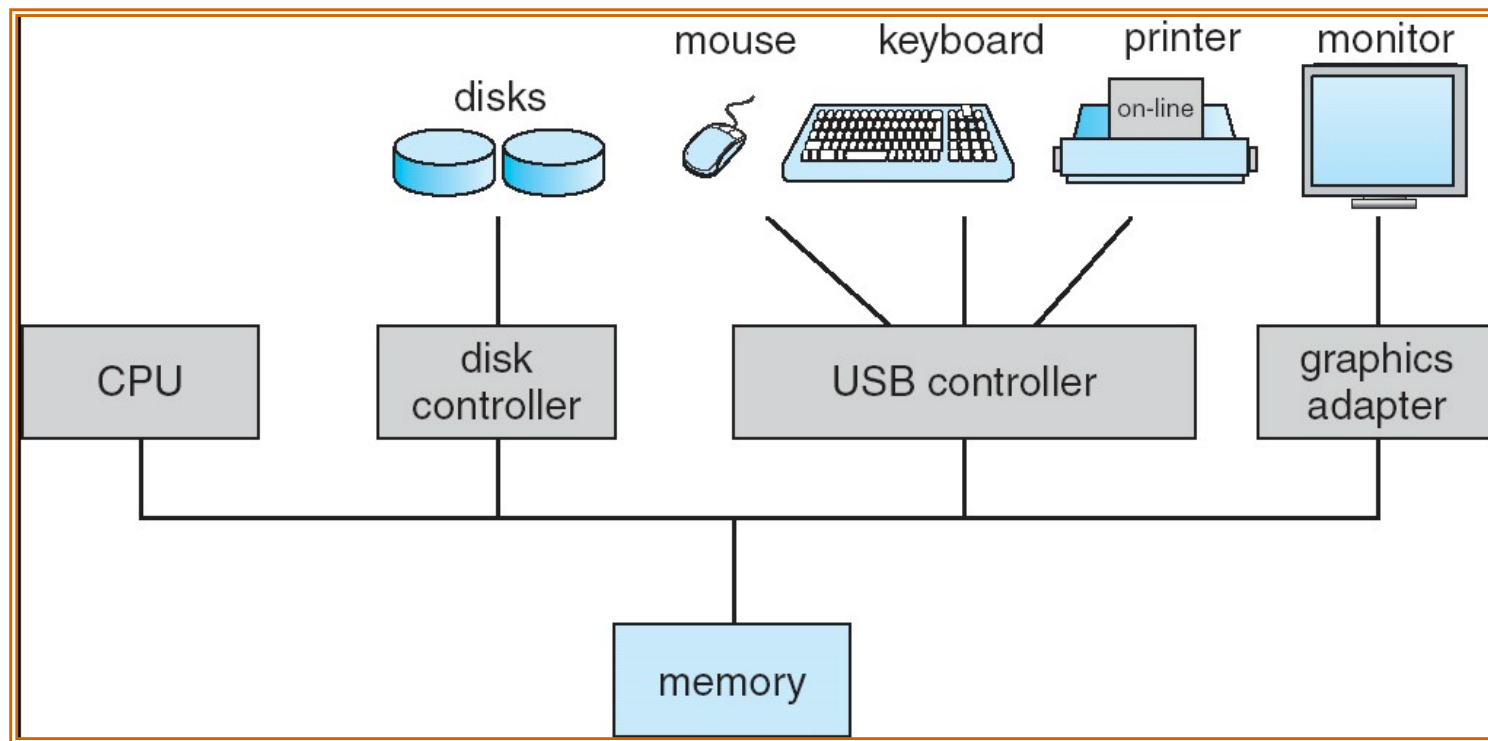
- ▶ I/O Communication Techniques

- ▶ **Computer Booting**

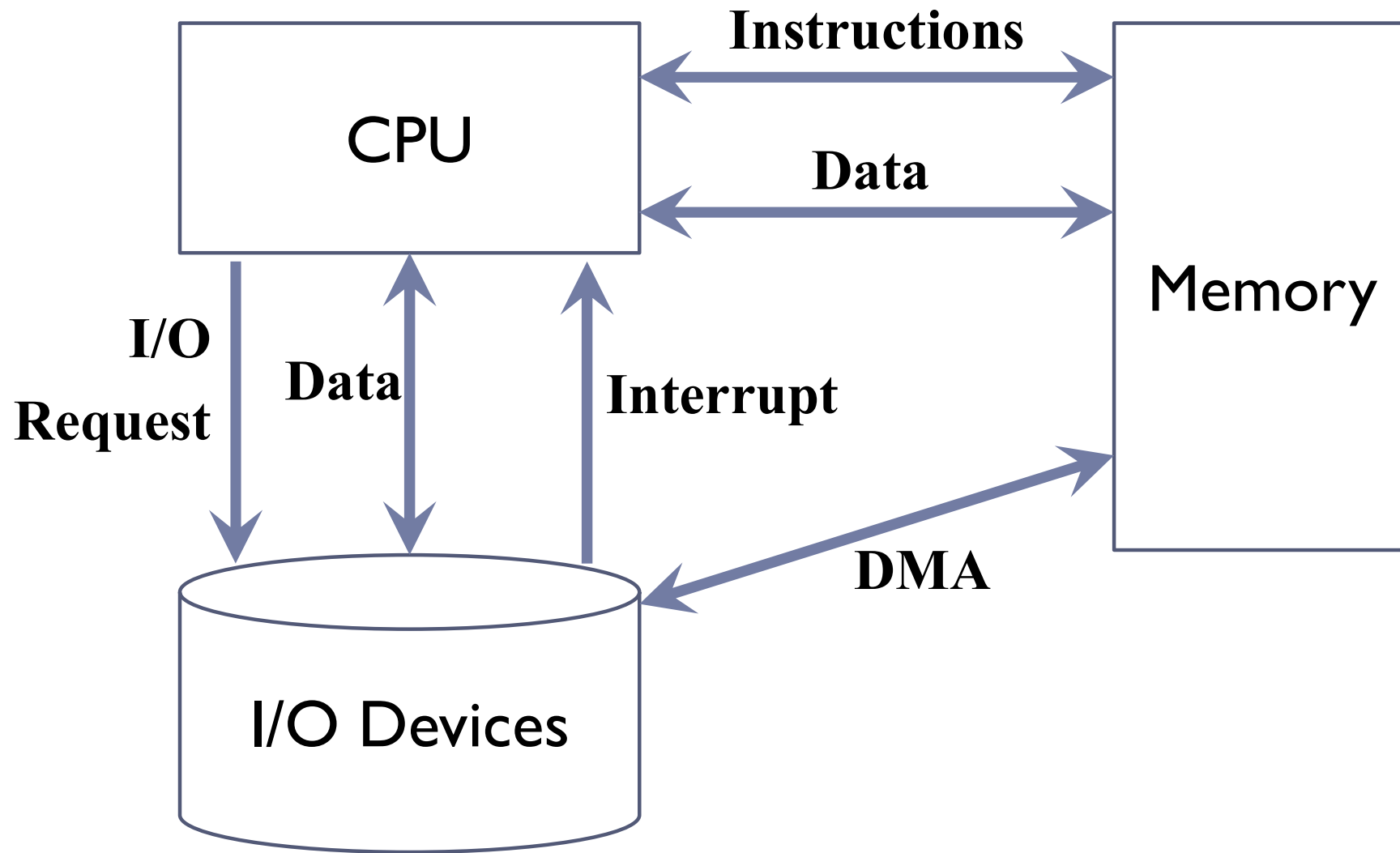


Basic Elements of a Computer System

A Simple Computer



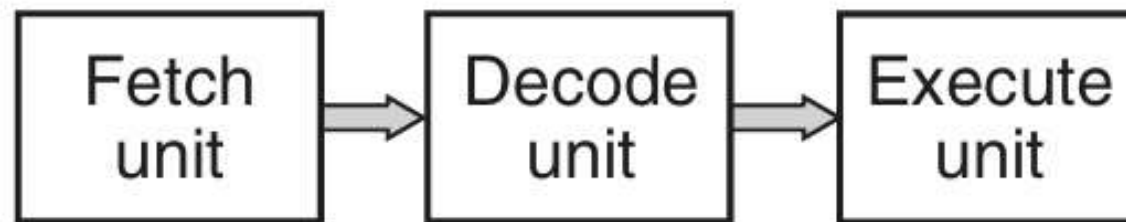
The CPU, memory, and I/O devices are connected by a system bus, and communicate with one another over it.



CPU_s

Processors

- ▶ The CPU is the “**brain**” of the computer
- ▶ The **basic cycle** of each CPU:
 - ▶ **(Fetch Phase)** fetch an instruction from memory
 - ▶ **(Execute Phase)** decode it and execute it.



This cycle is repeated until the program finishes

查徯曼凸淫拇产含曠

CPU违轹・庫拇 x 讎曠 x 截蟒：癩彭獲含曠〰枷紕學珏种懔拇产〰辞吳譟
竿曼轆蟒癩垚砂餘耘〰硃俠亾剂駝癩洄輯

CPU Registers (1)

处理器寄存器

- ▶ **A processor provides a set of registers**
 - ▶ It provides a level of memory faster and smaller than main memory
- ▶ **User-visible registers**
 - ▶ To store key data and
 - ▶ Enable programmer to minimize main-memory references by optimizing register use
- ▶ **Control and status registers**
 - ▶ Used by processor to control operating of the processor
 - ▶ Used by privileged, operating-system routines to control the execution of programs



Processor Registers (2)

User-Visible Registers

- ▶ Available to all programs - application programs and system programs
- ▶ Types of registers
 - ▶ Data Registers
 - ▶ They can be used with any machine instruction that performs operations on data
 - ▶ Restrictions: some dedicated registers may be used for floating-point operations and others for integer operations
 - ▶ Address Registers: Contain main memory addresses of data and instructions, or they contain a portion of the address that is used in the calculation of the complete address.



Processor Registers (3)

User-Visible Registers

▶ Address Registers

▶ Index (索引寄存器)

- ▶ involves adding an index to a base value to get an address

▶ Segment pointer (段指针)

- ▶ when memory is divided into segments, memory is referenced by a segment and an offset

▶ Stack pointer (栈指针)

- ▶ points to top of stack



Processor Registers (4)

Control and Status Registers

- ▶ **Program Counter (PC) 程序计数器**
 - ▶ Contains the address of the next instruction to be fetched
- ▶ **Instruction Register (IR) 指令寄存器**
 - ▶ Contains the instruction most recently fetched
- ▶ **Stack Pointer (SP) 栈指针**
 - ▶ Points to the top of the current stack in memory
- ▶ **Program Status Word (PSW) 程序状态字**
 - ▶ condition code bits
 - ▶ Interrupt enable or disable
 - ▶ execution mode: kernel or user mode

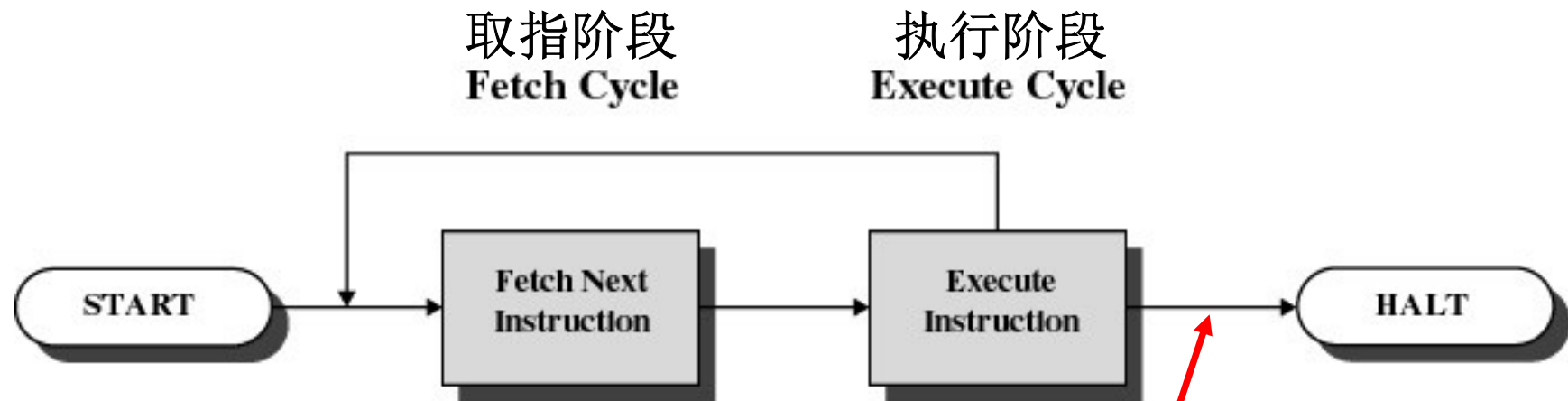


Instruction Execution (1)

Basic Instruction Cycle (基本指令周期)

In its **simplest** form, instruction processing consists of two steps:

The processor **reads (fetches)** instruction from memory one at a time and **executes** each instruction



Program halts only if the machine is turned off, some sort of unrecoverable error occurs, or a program instruction that halts the computer is encountered

Figure 1.2 Basic Instruction Cycle

Instruction Execution (2)

Instruction Fetch

- ▶ *The processor fetches the instruction from memory*
 - ▶ *Program counter (PC程序计数器) is used to get the address of the instruction to be fetched next*
 - ▶ *Fetches instruction is placed in the instruction register (IR指令寄存器)*



Instruction Execution (3)

Types of Instructions

- ▶ ***The actions required by instructions fall into four categories:***
 - ▶ ***Processor-memory: transfer data between processor and memory***
 - ▶ ***Processor-I/O: data transferred to or from a peripheral device***
 - ▶ ***Data processing: arithmetic or logic operation on data***
 - ▶ ***Control: alter sequence of execution***
- ▶ ***An instruction's execution may involve a combination of these actions***



Execution Modes

- ▶ **Most CPUs have two modes: kernel mode and user mode.**
 - ▶ The operating system runs in kernel mode, the CPU can execute every instruction in its instruction set and use every feature of the hardware
 - ▶ User programs runs in user mode, which permits only a subset of the instructions and a subset of features to be accessed.
- ▶ **Generally, all instructions involving I/O and memory protection are disallowed in user mode.**



Interrupts 中断



Interrupt Mechanism

- ▶ **Interrupt mechanism:** allow other modules (I/O, memory) to interrupt the normal execution sequence of the processor.
- ▶ **Goal:** to improve processing efficiency
 - ▶ Allows the processor to execute other instructions while an I/O operation is in progress
- ▶ A suspension of a process caused by an event external to that process and performed in such a way that the process can be resumed



Classes of Interrupts

Program	Generated by some condition that occurs as a result of an instruction execution, such as arithmetic overflow, division by zero, attempt to execute an illegal machine instruction, or reference outside a user's allowed memory space.
Timer	Generated by a timer within the processor. This allows the operating system to perform certain functions on a regular basis.
I/O	Generated by an I/O controller, to signal normal completion of an operation or to signal a variety of error conditions.
Hardware failure	Generated by a failure, such as power failure or memory parity error.



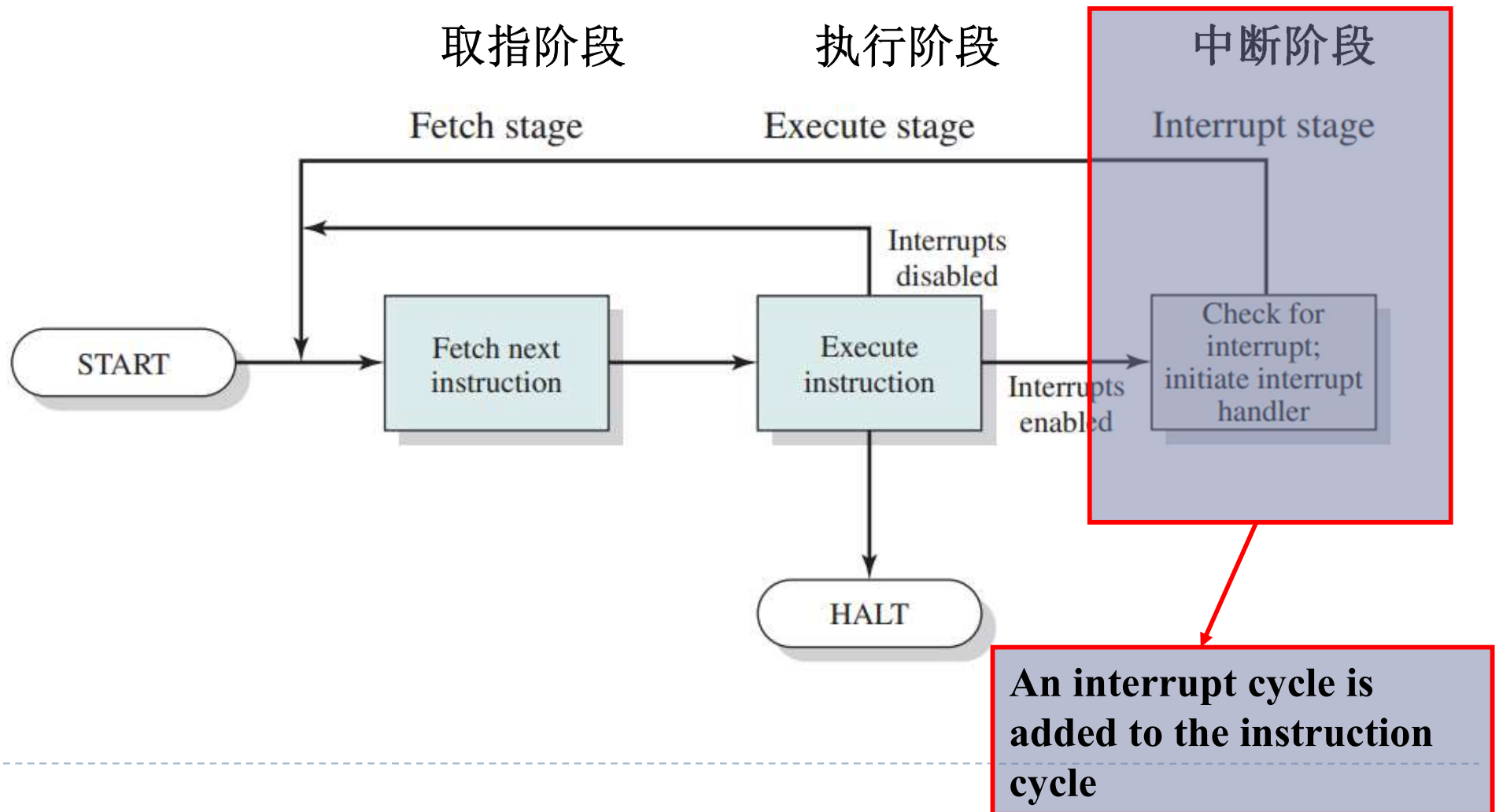
Interrupt Handler

- ▶ **A program that determines nature of the interrupt and performs whatever actions are needed**
- ▶ **Control is transferred to this program**
- ▶ **Generally part of the operating system**



Instruction Cycle with Interrupts

具有中断的指令周期



Instruction Cycle with Interrupts

Interrupt Cycle(中断阶段)

- ▶ **Processor checks for interrupts**
 - ▶ If no interrupts fetch the next instruction for the current program
 - ▶ If an interrupt is pending, suspend execution of the current program, and execute the interrupt handler



A Simple Procedure for Interrupt Handling

Handler:

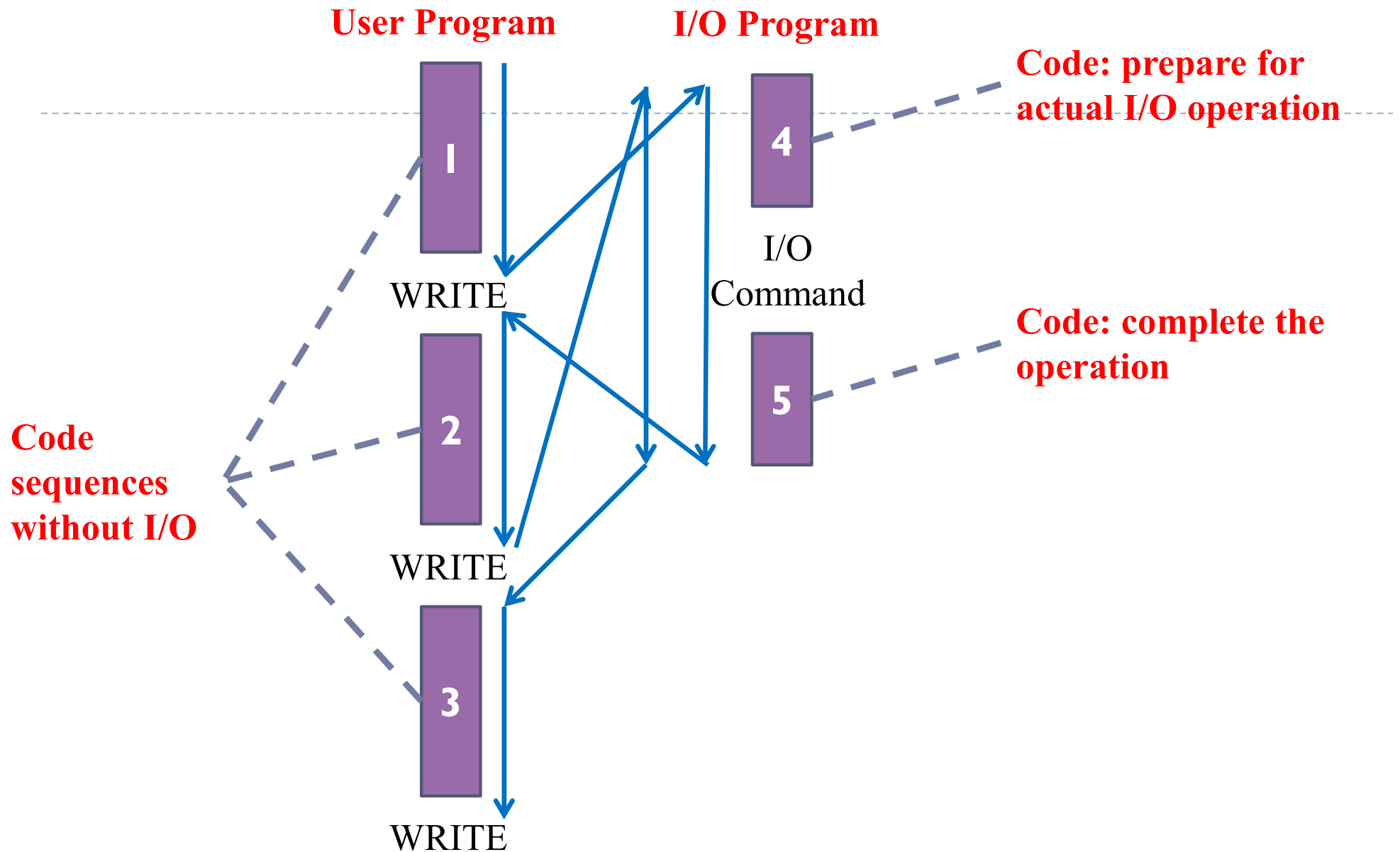
Save processor registers

execute device code

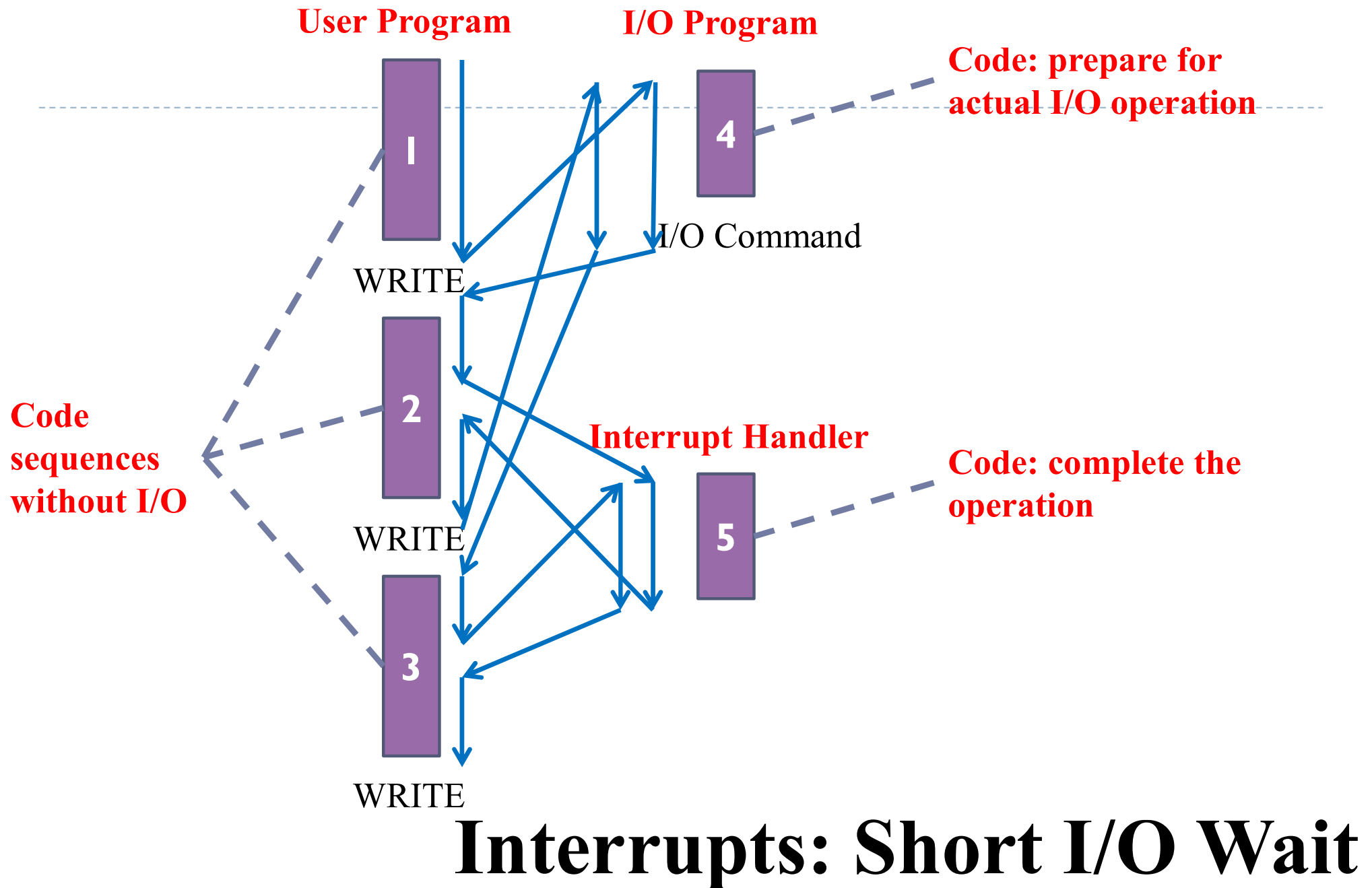
restore processor registers

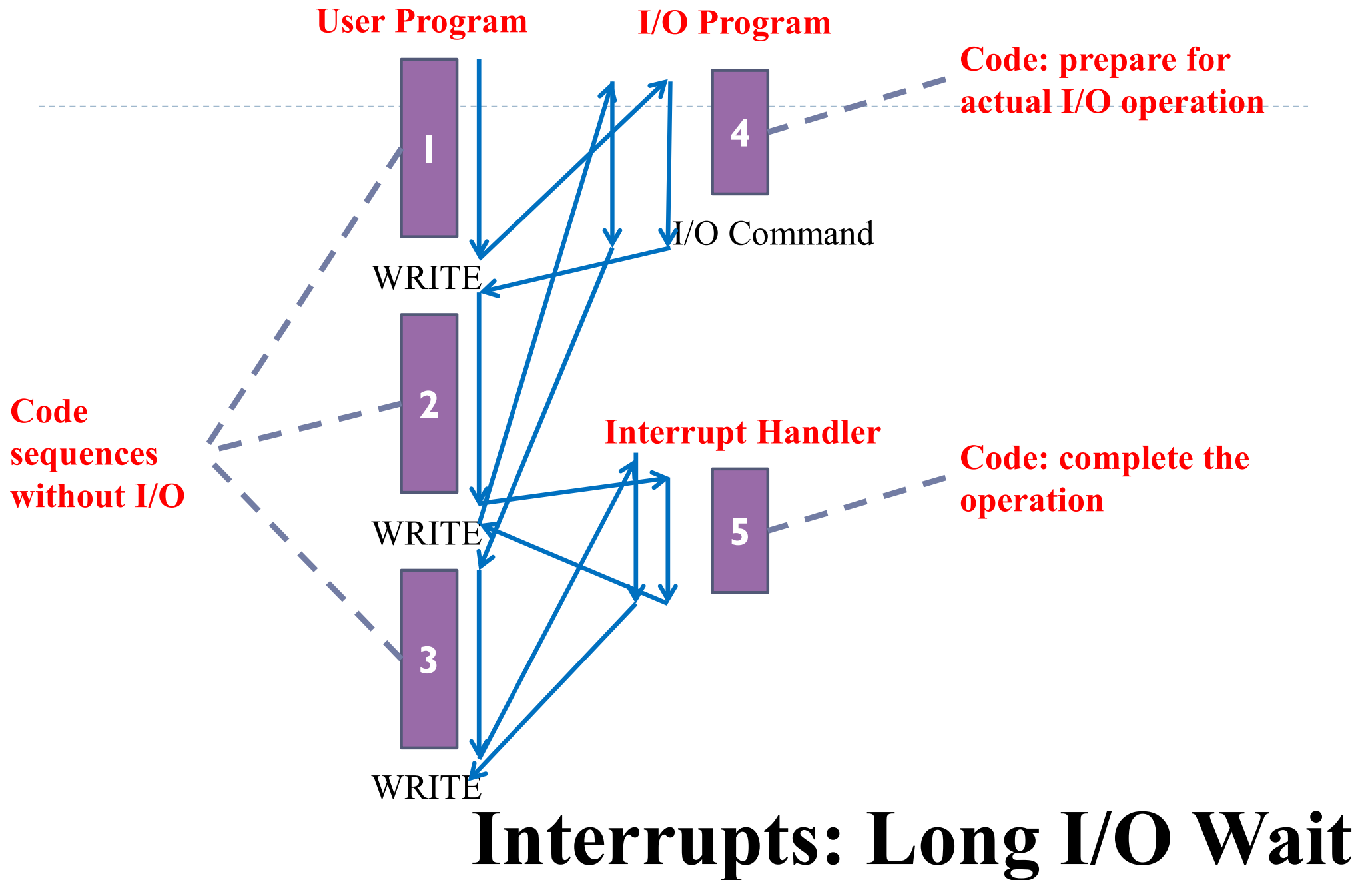
return to original program



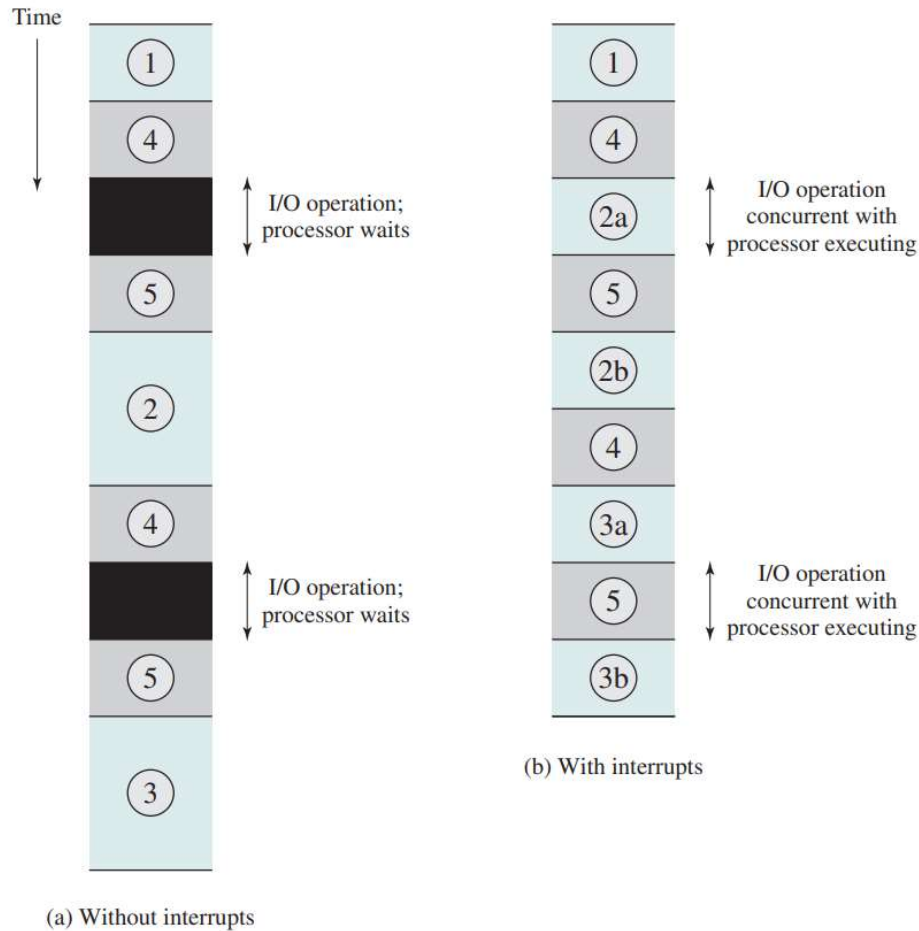


No Interrupts

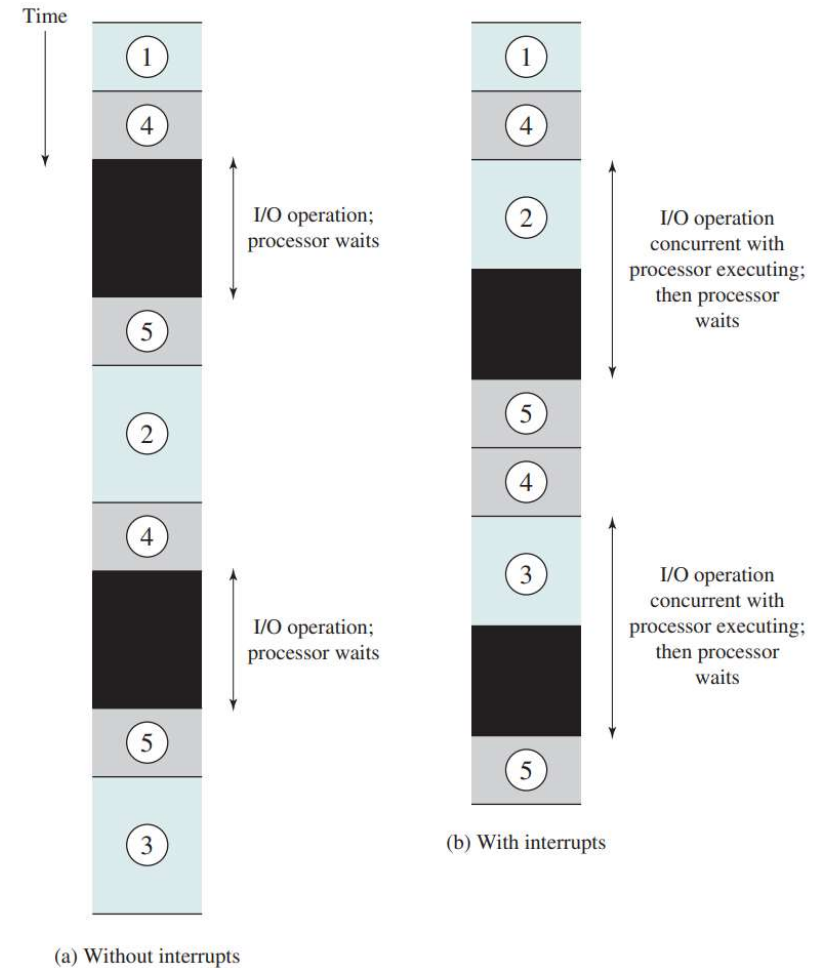




中断：程序时序

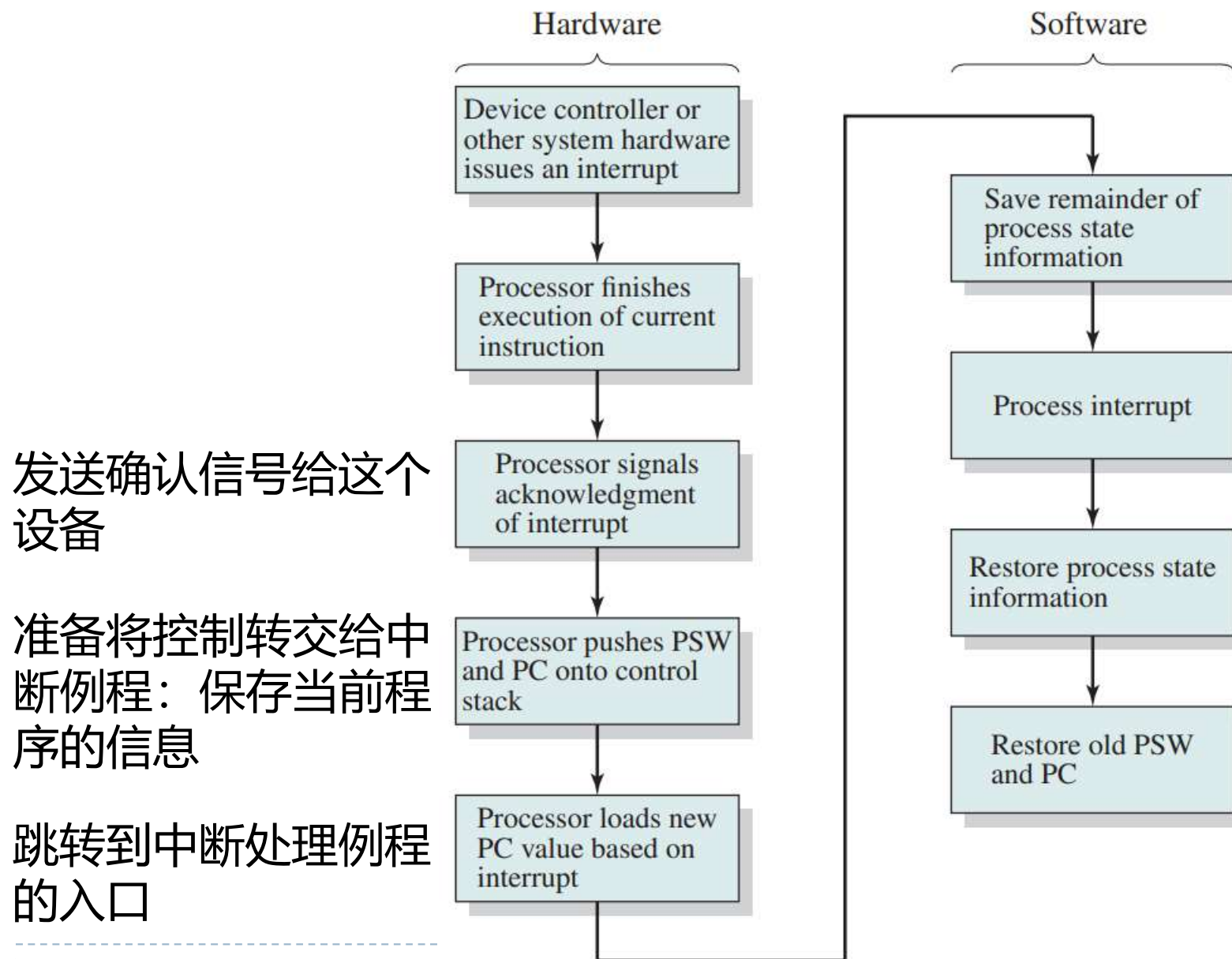


Short I/O Wait



Long I/O Wait

Interrupt Processing



发送确认信号给这个设备

准备将控制转交给中断例程：保存当前程序的信息

跳转到中断处理例程的入口

Multiple Interrupts (1)

- ▶ ***There are multiple I/O devices in the system***
 - ▶ ***Multiple interrupts can occur***
- ▶ ***Example: A program may be receiving data from a communications line and printing results.***
 - ▶ ***It is possible for a communications interrupt to occur while a printer interrupt is being processed***
 - ▶ ***How to deal with this situation?***



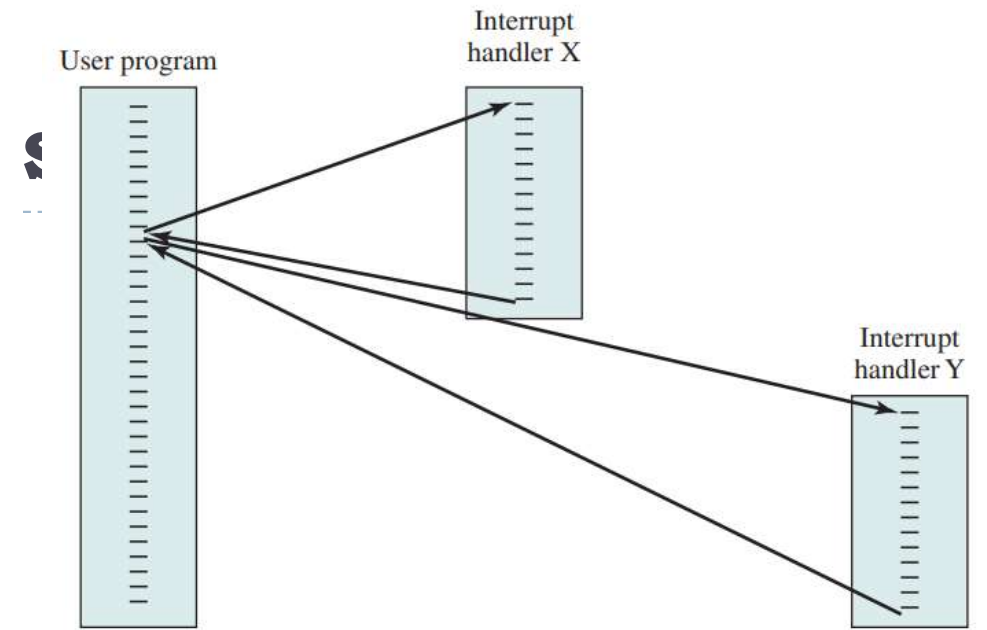
Multiple Interrupts (2)

Two approaches

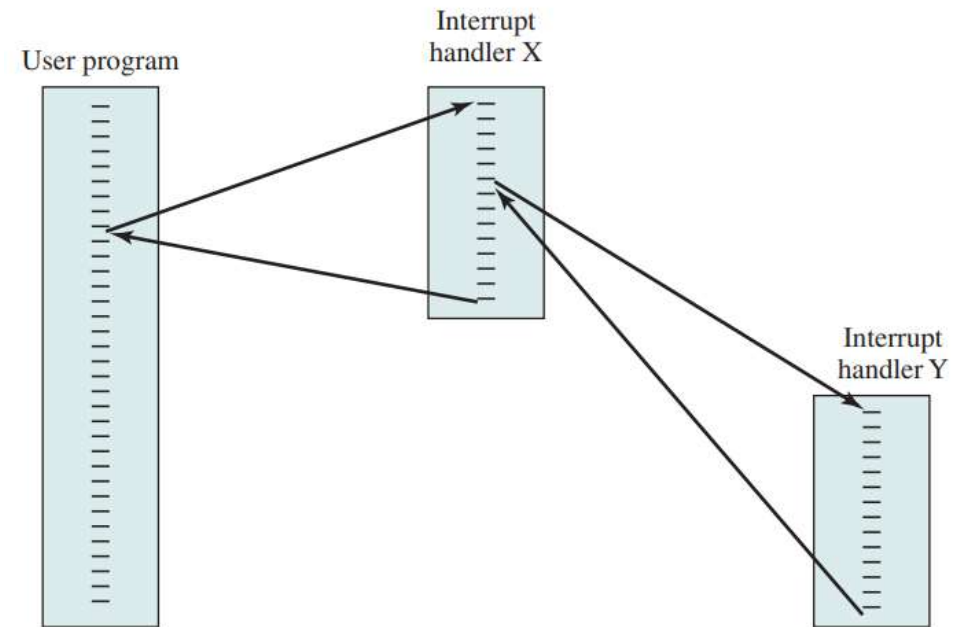
- ▶ **One approach:** *To disable interrupts while an interrupt is being processed*
 - ▶ **Drawback:** *It does not take into consideration **relative priority** or **time-critical needs***
- ▶ **The other approach:** *To define priorities for interrupts*
 - ▶ *To allow an interrupt of higher priority to interrupt an interrupt handler of lower priority.*



- *Disable interrupts so processor can complete the interrupt-handler*
- *Interrupts remain pending until the processor enables interrupts*
- *After interrupt handler routine completes, the processor checks for additional interrupts*
- *A higher priority interrupt can cause a lower-priority interrupt handler to be interrupted*
- *Example: when input arrives from communication line, it needs to be absorbed quickly to make room for more input*



(a) Sequential interrupt processing



(b) Nested interrupt processing

Multiprogramming

- ▶ **Processor has more than one program to execute**
- ▶ **The sequence the programs are executed depend on their relative priority and whether they are waiting for I/O**
- ▶ **After an interrupt handler completes, control may not return to the program that was executing at the time of the interrupt**



The Memory Hierarchy

存储层次结构

Memory 存储

- ▶ **Memory** is the second major component in any computer, to store data or information
- ▶ Ideally, a memory should be
 - ▶ **Speed**: Extremely fast (速度)
 - ▶ **Capacity**: Abundantly large (容量)
 - ▶ **Price**: Dirt Cheap (价格)
- ▶ It is hard to satisfy all these goals!! **How to do with it?**

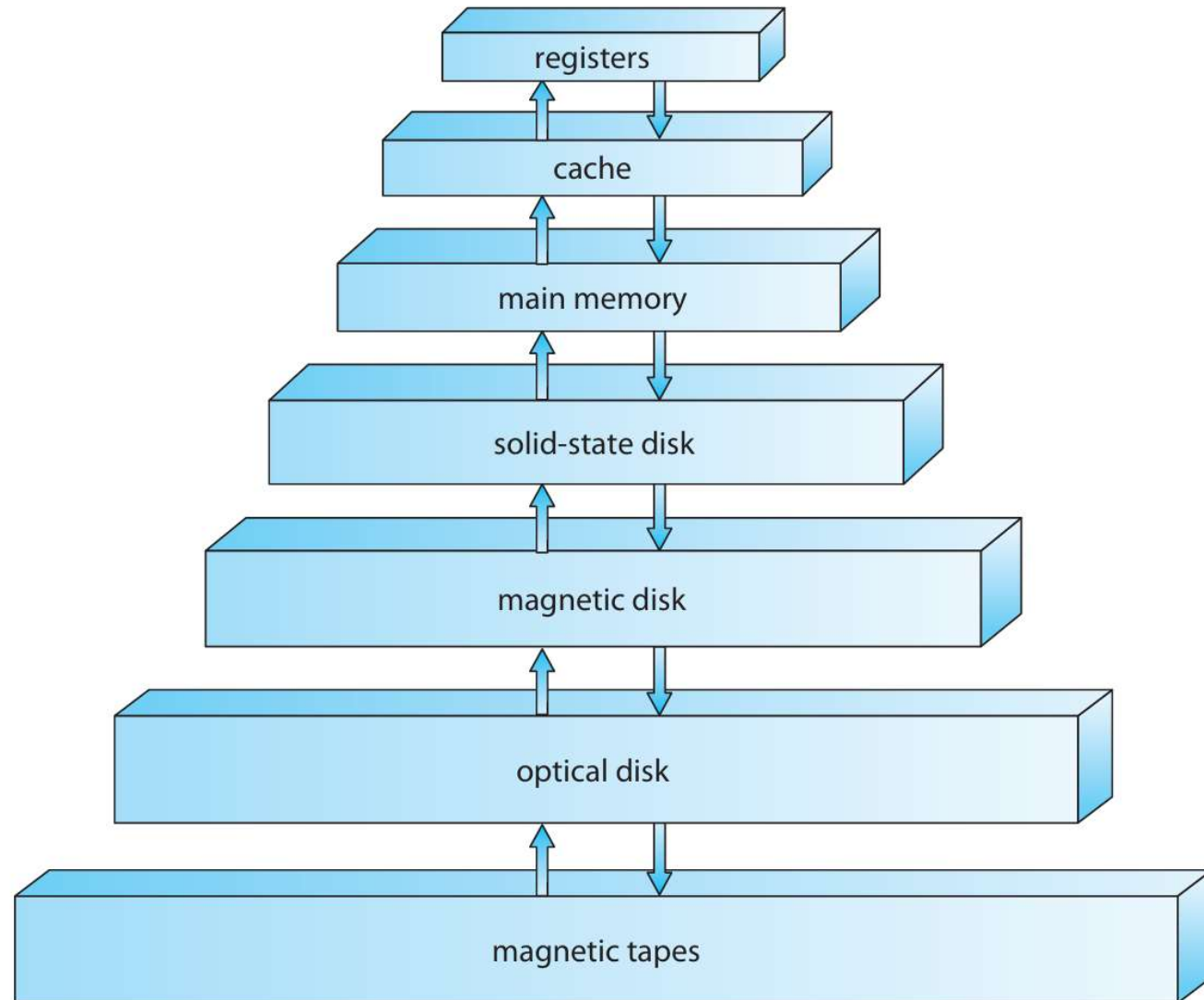
Memory Hierarchy



Memory Hierarchy

存储层次

Increasing capacity
Decreasing access speed
Decreasing cost per bit
Decreasing access frequency by the processor



Registers

- ▶ **They are internal to the CPU**
 - ▶ They are as fast as CPU
 - ▶ Capacity: typically 32*32-bits on a 32-bit CPU and 64*64-bits on a 64-bit CPU.
 - ▶ Programs must manage the registers (i.e. decide what to keep in them) themselves, in software.



Main Memory

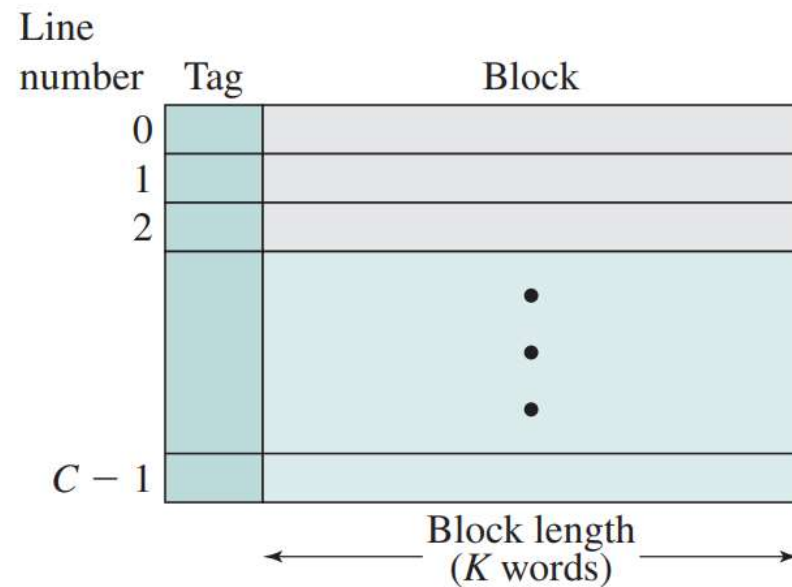
- ▶ Main memory is often called RAM (Random Access Memory)
- ▶ All CPU requests that can not be satisfied out of the cache go to main memory
- ▶ The **key to the success** of cache memory is “decreasing frequency of memory access”



Cache Memory

- ▶ Mostly controlled by the **hardware**
- ▶ **Cache Line (缓存行)**
 - ▶ Main Memory is divided into **cache lines or blocks** (typically 64 bytes): addresses 0 to 63 in cache line 0, addresses 64 to 127 in cache line 1, and so on.
 - ▶ The most heavily used cache lines are kept in a high-speed cache located inside or very close to the CPU
- ▶ **Cache Hit (缓存命中)**
 - ▶ when the program needs to read a memory word, the cache hardware finds out that the line is in the cache
- ▶ Capacity is limited, due to its high cost





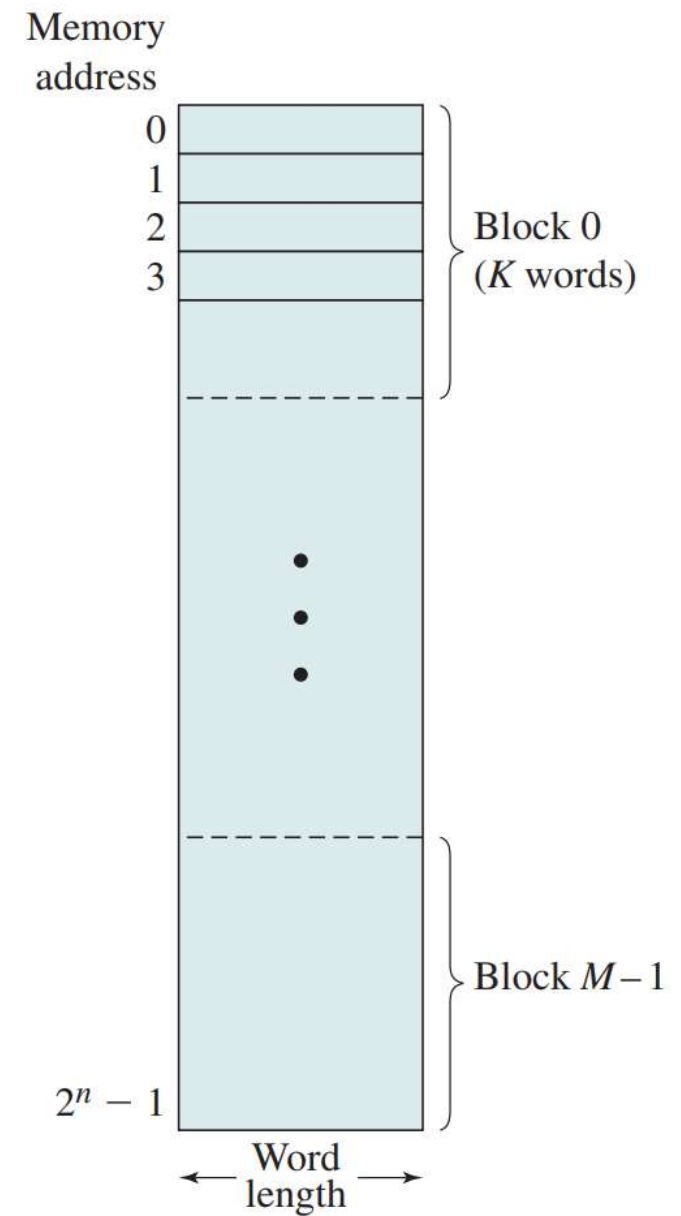
(a) Cache

The number of cache slots: C

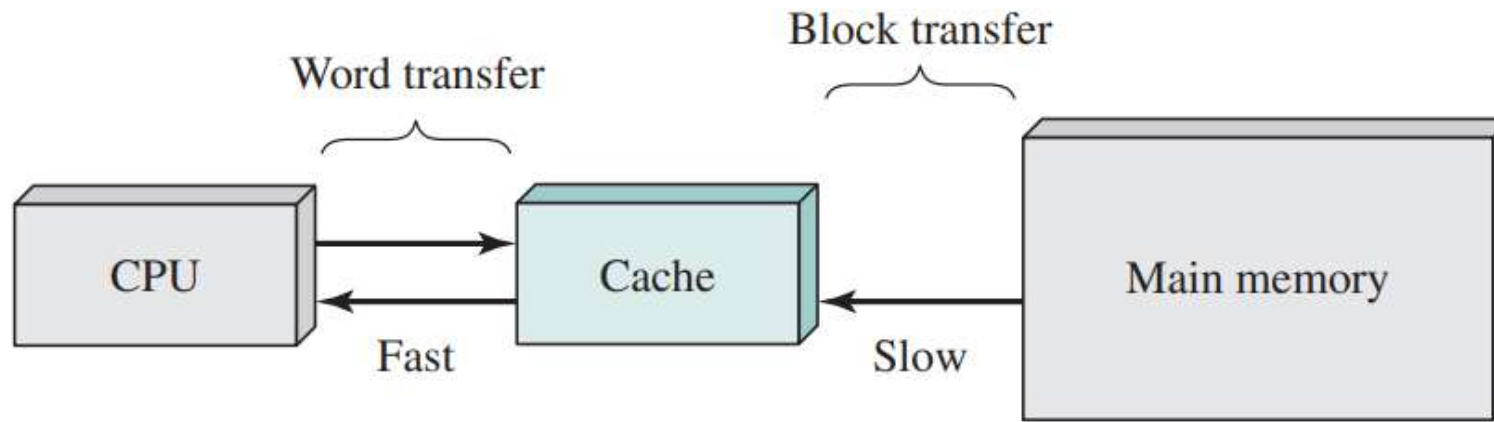
The number of memory blocks: $M = \frac{2^n}{K}$

$$C \ll M$$

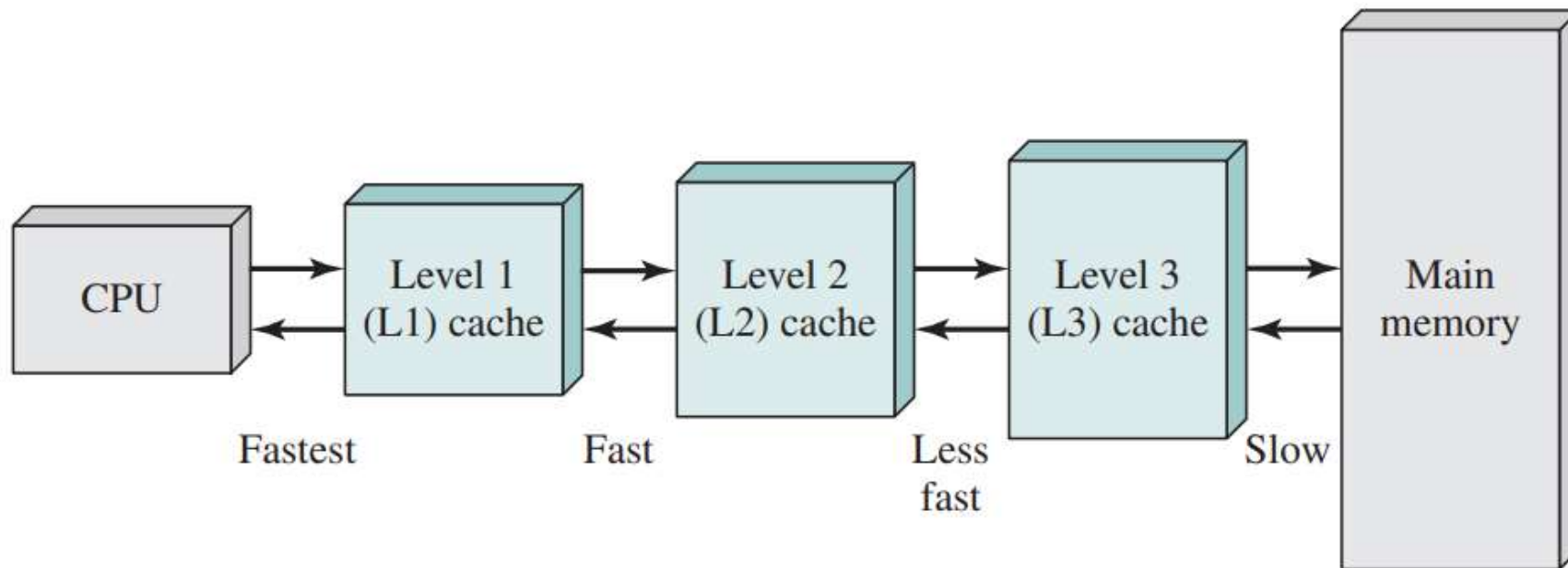
Each cache slot includes a tag to indicate which memory block is currently stored there.



(b) Main memory



(a) Single cache



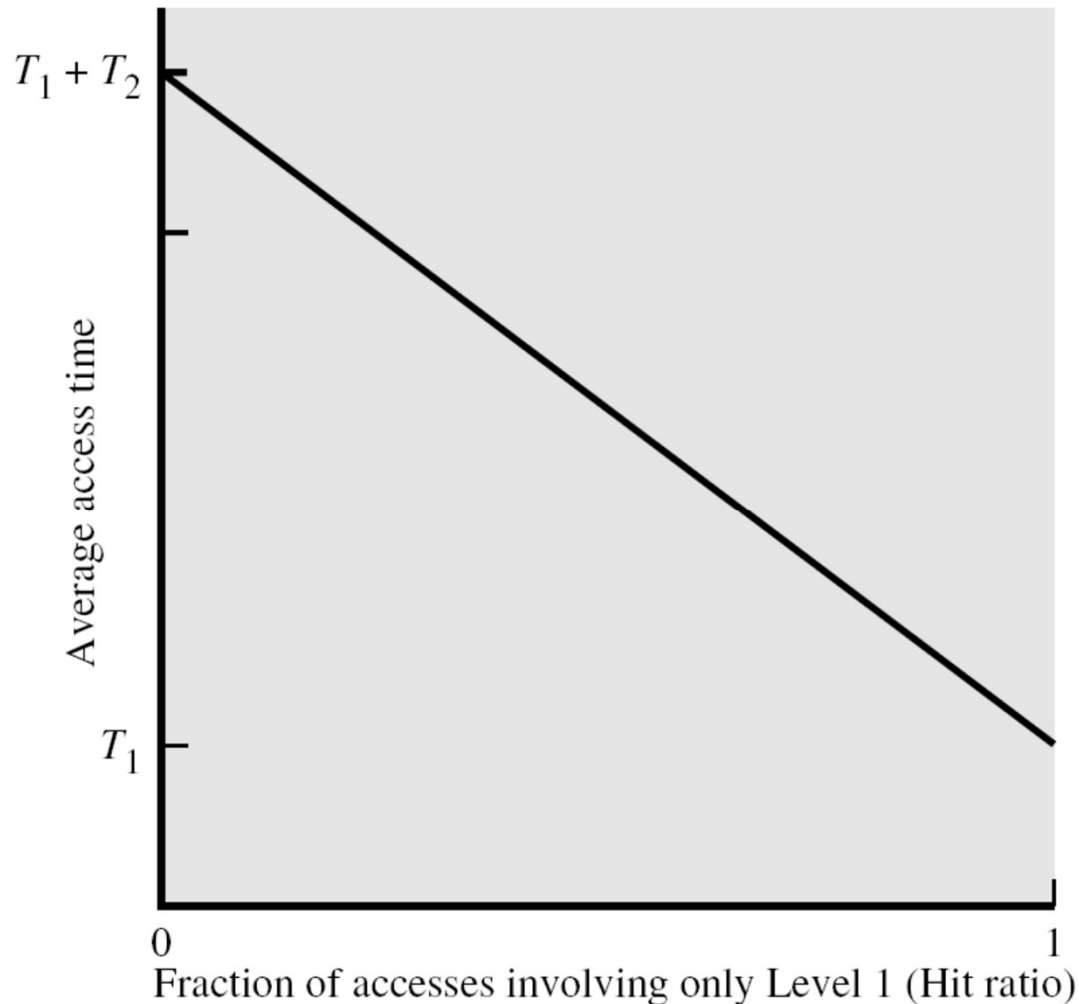
(b) Three-level cache organization

Hit Ratio of Cache

- ▶ **Benefits of cache memory**
 - ▶ Invisible to operating system
 - ▶ Increase the speed of memory
- ▶ **Hit Ratio**
 - ▶ Hit ratio is defined as the fraction of all memory accesses that are found in the faster memory (cache memory)



Performance of a Simple Two-Level Memory



- ▶ T_1 → the access time to level 1
- ▶ T_2 → the access time to level 2

If hit ratio is 95%, $T_1 = 0.1\mu\text{s}$, and $T_2 = 1\mu\text{s}$, then the average time to access a word can be expressed as

$$0.95 \cdot 0.1 + 0.05 \cdot (0.1 + 1) = 0.15\mu\text{s}$$

The basis for the success

- ▶ Principle: Locality of reference
 - ▶ During the course of execution of a program, memory references by the processor tend to cluster.
- ▶ This principle can be applied across more than two levels of memory.
 - ▶ Exercise 1.13 → Assignment!!



Example: Intel Core i7 Xeon 5500

Intel Core i7 Xeon 5500 at 2.4 GHz

Memory	Access time	Size
register	1 cycle	64 bytes
L1 cache	~4 cycles	64 kilobytes
L2 cache	~10 cycles	4 megabytes
L3 cache	~40-75 cycles	8 megabytes
remote L3	~100-300 cycles	
Local DRAM	~60 nsec	
Remote DRAM	~100 nsec	



Design of Cache Memory (1)

高速缓冲存储器设计

- ▶ **Design issues include:**
 - ▶ **Cache size (高速缓冲存储器大小)**
 - ▶ **Block size (块大小)**
 - ▶ **Mapping function (映射函数)**
 - ▶ **Replacement algorithm (替换算法)**
 - ▶ **Write policy (写策略)**



I/O Devices

A computer is of no use without some means of getting data into it and get information out of it.

I/O Devices

- ▶ **I/O devices generally consist of two parts:**
 - ▶ *A controller*
 - ▶ The device itself
- ▶ **The job of the controller is to present a *simpler interface* to the OS**
 - ▶ The actual device interface is hidden behind the controller



Device Drivers

- ▶ ***Each type of controller is different, so different software is needed to control each one.***
 - ▶ *Device driver is the software that talks to a controller, giving it commands and accepting responses.*
- ▶ ***Each controller manufacturer has to supply a driver for each operating system it supports***



Device Registers

- ▶ **Every controller has a small number of registers for communication with Processors**
- ▶ **I/O addressing**
 - ▶ **Special instructions: Special IN and OUT instructions are available in kernel mode to allow drivers to read and write the registers**
 - ▶ **Memory-mapped I/O: The device registers are mapped into the operating system's address space**



I/O Communication Techniques

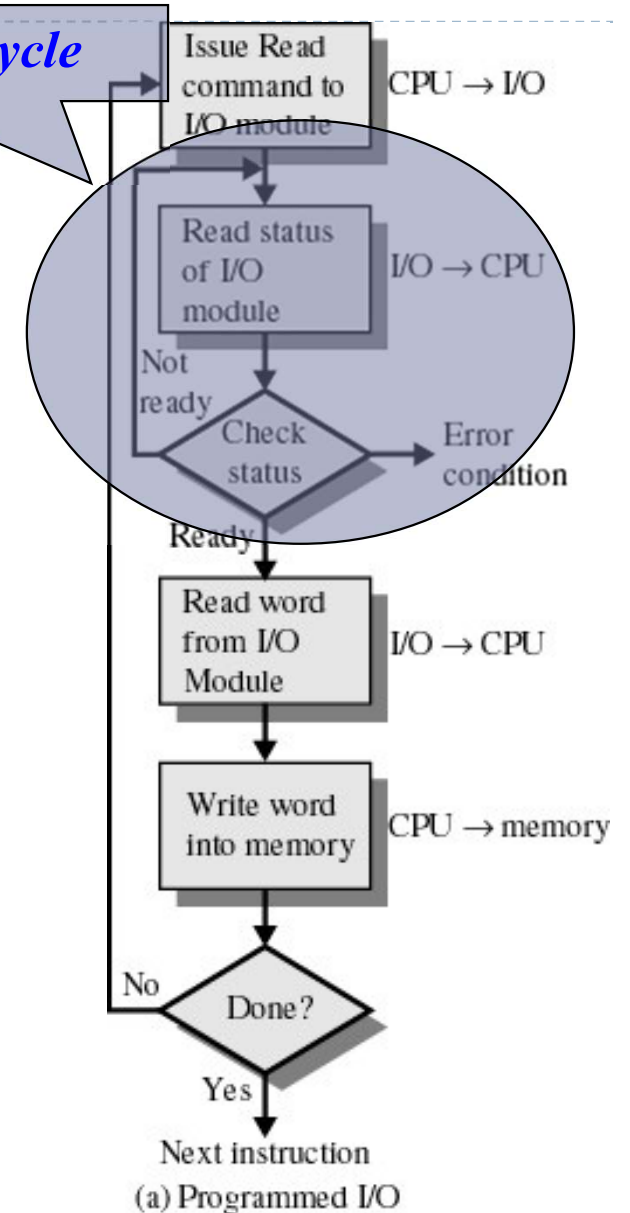
- ▶ **Programmed I/O**
可编程I/O
- ▶ **Interrupt-Driven I/O**
中断驱动I/O
- ▶ **Direct Memory Access (DMA)**
直接内存存取



Programmed I/O

- ▶ *I/O module performs the action, not the processor*
- ▶ Sets appropriate bits in the I/O status register
- ▶ No interrupts occur
- ▶ Processor checks status until operation is complete

Status checking cycle

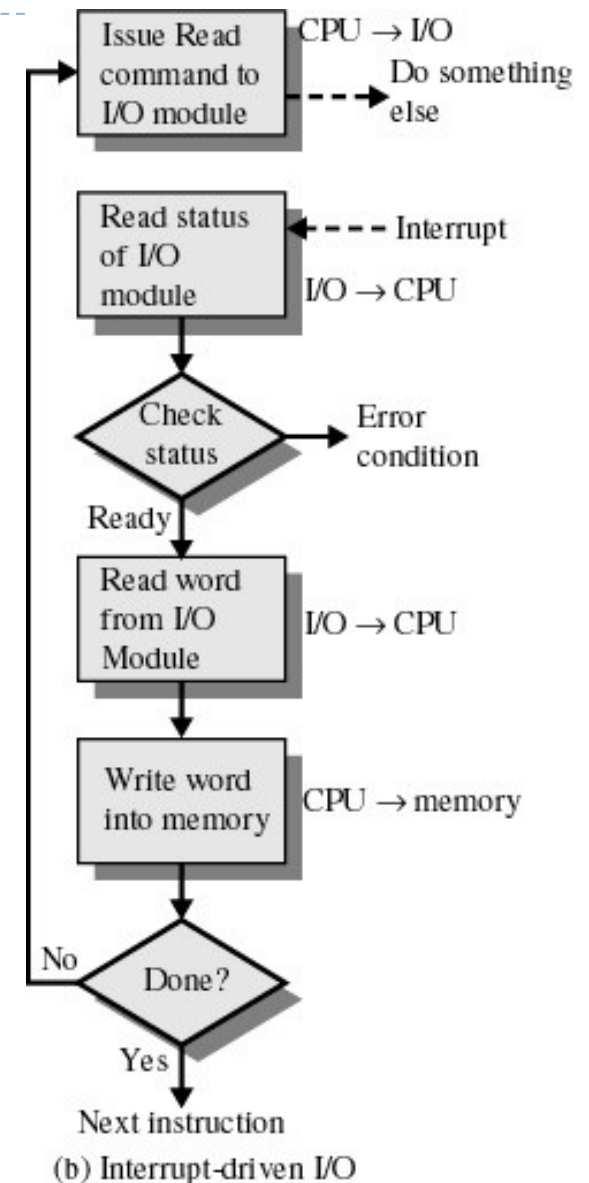


-
- ▶ With programmed I/O, the problem is that the processor has to wait a long time for the I/O module of concern to be ready for either reception or transmission of the I/O module.



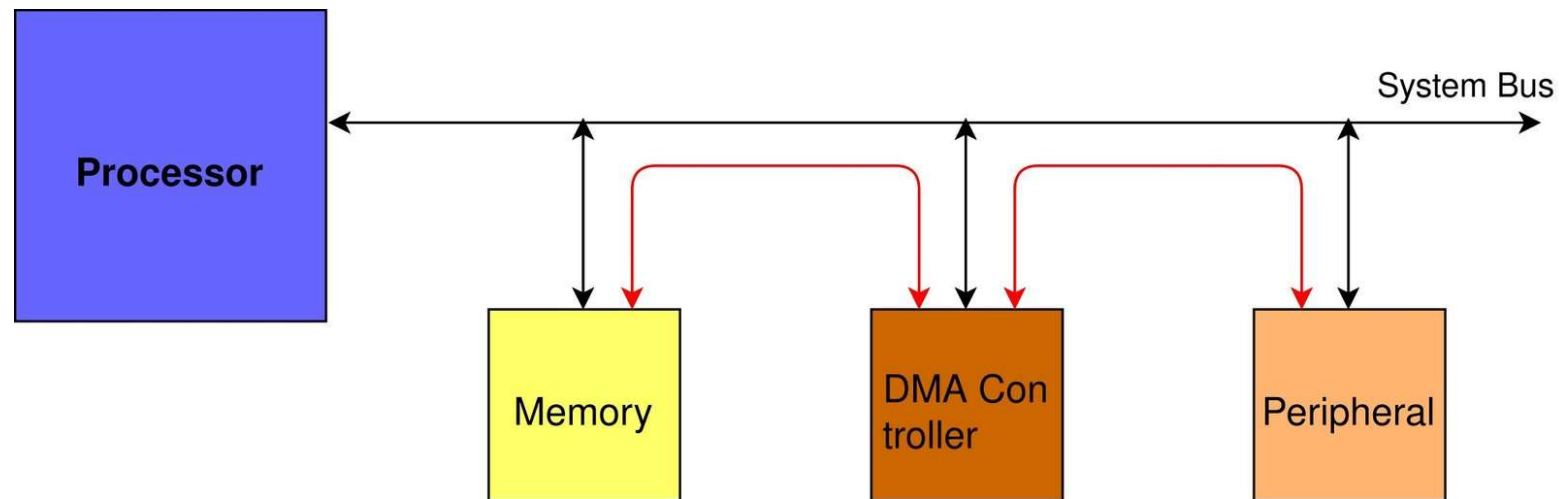
Interrupt-Driven I/O

- ▶ Processor is interrupted when I/O module ready to exchange data
- ▶ Processor is free to do other work
- ▶ No needless waiting
- ▶ Consumes a lot of processor time because every word read or written passes through the processor

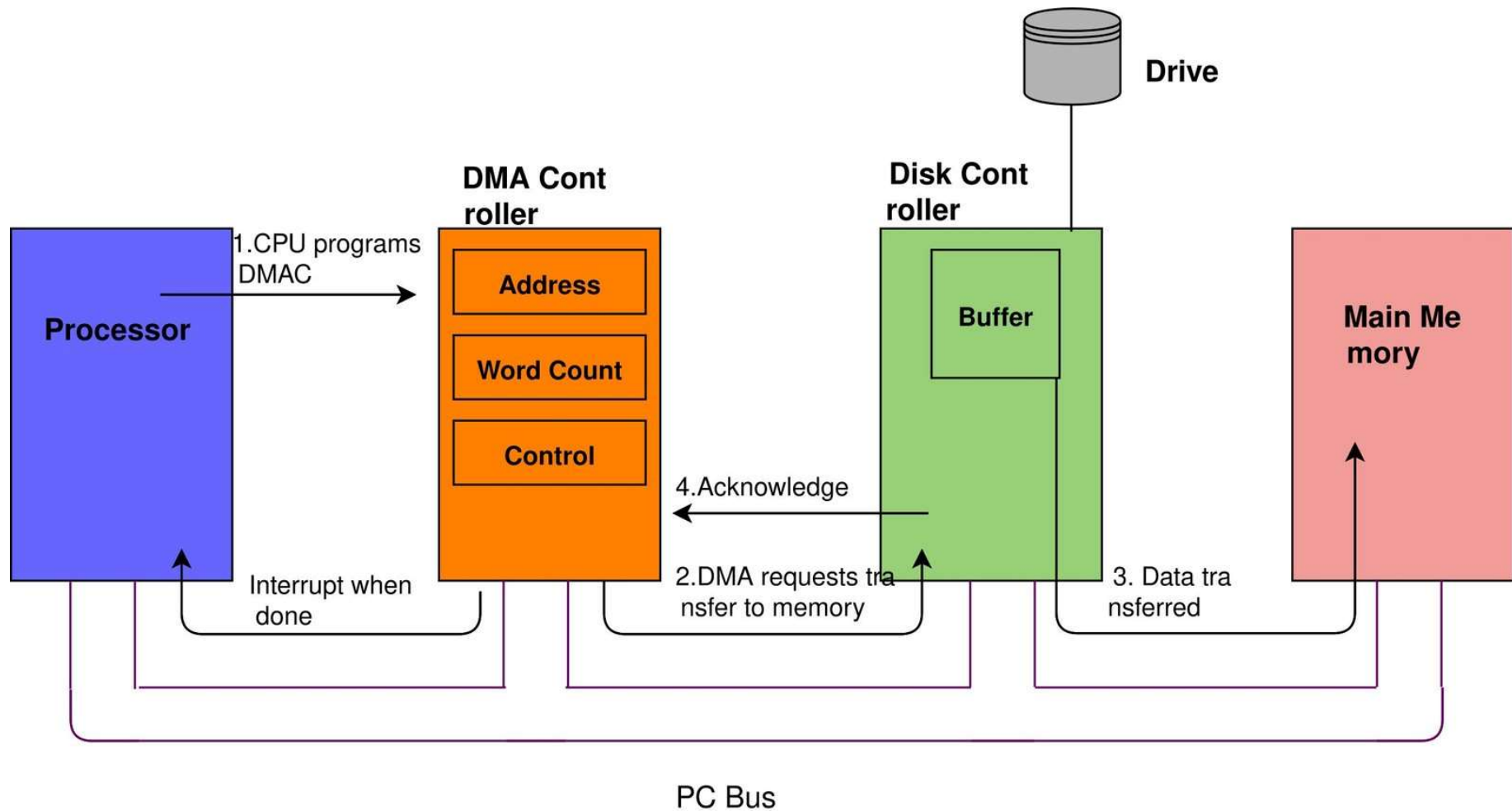


Direct Memory Access (DMA)

- ▶ **Direct Memory Access is a data transfer technique**
 - ▶ I/O Devices communicate directly with the memory without passing through CPU
 - ▶ A DMA controller is responsible for accessing I/O devices and memory for transferring data

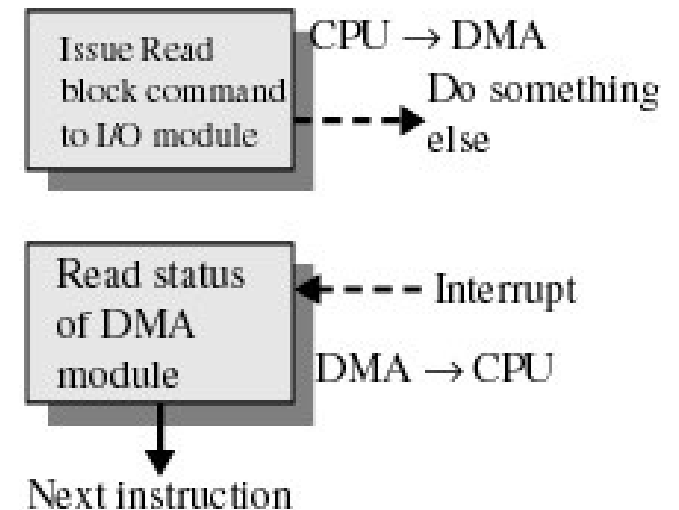


Direct Memory Access (DMA)



Direct Memory Access (1)

- ▶ DMA chip transfers a **block of data** between memory and some device controller
 - ▶ Without constant CPU intervention
- ▶ An **interrupt** is sent when the task is complete
- ▶ The CPU is only involved at the beginning and end of the transfer



(c) Direct memory access

Direct Memory Access (2)

- ▶ **The information that should be sent to the DMA module**
 - ▶ Whether a read or write is requested
 - ▶ The address of the I/O device involved
 - ▶ The starting location in memory to read from or write to
 - ▶ The number of words to be read or written



System Booting

系统启动

Booting the Computer

(1)

- ▶ Every Pentium contains a parentboard (or motherboard).
 - ▶ On the parentboard is a program called the system BIOS (Basic Input Output System)
 - ▶ The BIOS contains low-level I/O software



Booting the Computer

(2)

- ▶ When the computer is booted, the BIOS is started.
 - ▶ It first checks to see how much RAM is installed and whether the keyboard and other basic devices are installed and responding correctly
 - ▶ It starts out by scanning the ISA and PCI buses to detect all the devices attached to them
 - ▶ The BIOS then determine the boot device by trying a list of devices stored in the CMOS memory
 - ▶ This loader reads in the operating system from the active partition and starts it.



Booting the Computer

- ▶ Then what?
 - ▶ The first block (called **boot block**) from the boot device is read into memory and executed
 - ▶ This sector contains a program that normally examines the partition table at the end of the boot sector to determine **which partition is active**.
 - ▶ Then a **secondary boot loader** is read in from that partition. This loader reads in the operating system from the active partition and starts it.
 - ▶ The operating system then queries the BIOS to get the configuration information. For each device, to see if it has the device driver.



How Linux Boot?

